Aula 4 - Solicitação de Empréstimo

Objetivos da aula

- Criar a tela de detalhes do EPI;
- Passar dados de uma tela para outra (navegação com parâmetros);
- Montar um formulário simples com validação;
- Criar um endpoint POST /emprestimos na API Java;
- Receber no @RequestBody a entidade Emprestimo com o funcionário e o EPI aninhados;
- Aplicar regras de negócio antes de gravar (CA vencido, estoque, pedido repetido);
- Gravar o empréstimo com status SOLICITADO, ligando funcionário e EPI.

Problema proposto

O capacete do Bruno (matrícula 1002, Almoxarifado) trincou. Hoje ele precisa ir até a Segurança do Trabalho e pedir outro no papel. Queremos que ele faça isso pelo app:

1. Faz login;
2. Abre a lista de EPIs e toca no capacete;
3. Vê os detalhes (CA, validade, quantidade disponível);
4. Escreve o motivo e toca em Solicitar Empréstimo.

A API grava o pedido na tabela `emprestimo` com status `SOLICITADO`. Nas próximas aulas o administrador vai aprovar o pedido e, só então, o EPI sai do estoque.

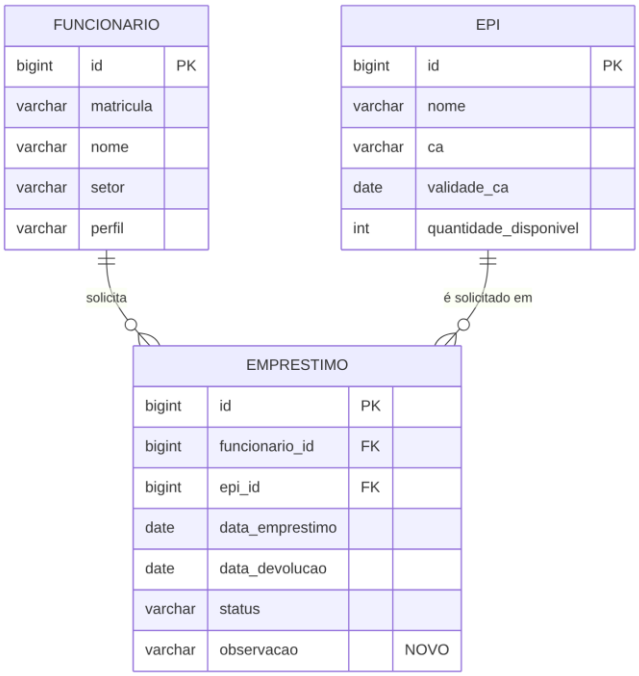
Por que isso importa? A NR-06 exige que a entrega de EPI ao trabalhador seja registrada. O pedido feito pelo app já deixa o rastro de quem pediu, o quê, quando e por quê.

Conceitos trabalhados

Conceito	Onde aparece nesta aula
Navegação com parâmetros	A lista de EPIs envia o EPI escolhido e o usuário logado para a tela de detalhes.
POST na API	O app envia um JSON para POST /emprestimos e a API devolve 201 (criado) ou 400 (erro).
Formulários simples	Form + TextFormField com validator para o motivo da solicitação.
Relacionamento entre tabelas	Cada empréstimo aponta para um funcionário e para um EPI (@ManyToOne).

Parte 1 - Preparando o banco de dados

A tabela `emprestimo` já existe desde a Aula 1. Ela é a "ponte" entre `funcionario` e `epi`:



- `funcionario_id` guarda o id de quem pediu → aponta para `funcionario.id`
- `epi_id` guarda o id do EPI pedido → aponta para `epi.id`
- Um funcionário pode ter muitos empréstimos, e um EPI pode aparecer em muitos empréstimos.

No Java isso é o `@ManyToOne` que você já escreveu na entidade Emprestimo.

a) Acrescentar o campo observacao na entidade

Arquivo: Emprestimo.java

Adicionar abaixo do campo status:

```
// NOVO na Aula 4: motivo informado pelo funcionário no formulário
@Column(length = 200)
private String observacao;
```

Como o application.properties está com `spring.jpa.hibernate.ddl-auto=update`, ao reiniciar a API o Hibernate cria a coluna sozinho. Se preferir criar pelo Workbench:

```
ALTER TABLE emprestimo ADD COLUMN observacao VARCHAR(200);
```

Obs: a coluna não tem nullable = false. Assim os registros antigos (se houver) não dão erro.

b) Conferir a entidade completa

Emprestimo.java

```java
package com.senac.api_epi.entities;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

import java.time.LocalDate;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Entity
public class Emprestimo {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(optional = false)
    @JoinColumn(name = "funcionario_id")
    private Funcionario funcionario;

    @ManyToOne(optional = false)
    @JoinColumn(name = "epi_id")
    private Epi epi;

    @Column(nullable = false)
    private LocalDate dataEmprestimo;

    private LocalDate dataDevolucao;

    @Column(nullable = false, length = 15)
    private String status;

    // NOVO na Aula 4: motivo informado pelo funcionário no formulário
    @Column(length = 200)
    private String observacao;
}
```

Etapa 2 – Como o app vai enviar o pedido

Na Aula 3, o AuthController recebeu a própria entidade Funcionario no `@RequestBody`. Agora vamos fazer igual: o controller vai receber a entidade **Emprestimo**.

Só que o Emprestimo não tem os campos funcionarioId e epiId. Ele tem um objeto `funcionario` e um objeto `epi` (o @ManyToOne). Por isso o JSON precisa ter o mesmo formato, com o funcionário e o EPI "dentro" do pedido, informando só o id:

```
{
  "funcionario": { "id": 2 },
  "epi": { "id": 1 },
  "observacao": "Capacete atual está trincado"
}
```

O Spring transforma esse JSON em um objeto Emprestimo. Dentro dele, o Funcionario e o Epi chegam com o id preenchido e o resto vazio (null). O Service vai usar esses ids para buscar os dados completos no banco.

JSON enviado	Entidade Java	Tabela no MySQL
`"funcionario": { "id": 2 }`	`Funcionario funcionario`	`funcionario_id = 2`
`"epi": { "id": 1 }`	`Epi epi`	`epi_id = 1`
`"observacao": "..."`	`String observacao`	`observacao`

Cuidado: nunca confie no que vem do app

Como recebemos a entidade inteira, alguém poderia enviar pelo Postman um "status": "EMPRESTADO" ou um "id" de outro registro. Por isso o Service vai sobrescrever esses campos antes de gravar. Em sistemas maiores, esse problema é resolvido com uma classe só para entrada de dados, chamada DTO (o pacote dto que criamos na Aula 1 existe para isso). Veremos mais adiante.

Etapa 3 – Criar o EmprestimoService

Pacote: `com.senac.api_epi.service`

Criar a classe: EmprestimoService

EmprestimoService.java

```java
package com.senac.api_epi.service;

import com.senac.api_epi.entities.Emprestimo;
import com.senac.api_epi.entities.Epi;
import com.senac.api_epi.entities.Funcionario;
import com.senac.api_epi.repositories.EmprestimoRepository;
import com.senac.api_epi.repositories.EpiRepository;
import com.senac.api_epi.repositories.FuncionarioRepository;
import org.springframework.stereotype.Service;

import java.time.LocalDate;
import java.util.List;

@Service
public class EmprestimoService {

    private final EmprestimoRepository emprestimoRepository;
    private final FuncionarioRepository funcionarioRepository;
    private final EpiRepository epiRepository;

    public EmprestimoService(EmprestimoRepository emprestimoRepository,
                             FuncionarioRepository funcionarioRepository,
                             EpiRepository epiRepository) {
        this.emprestimoRepository = emprestimoRepository;
        this.funcionarioRepository = funcionarioRepository;
        this.epiRepository = epiRepository;
    }
```

EmprestimoService.java

```java
    public Emprestimo solicitar(Emprestimo pedido) {

        // 1. O app enviou o funcionário e o EPI com id?
        if (pedido.getFuncionario() == null
                || pedido.getFuncionario().getId() == null
                || pedido.getEpi() == null
                || pedido.getEpi().getId() == null) {
            throw new IllegalArgumentException("Informe o funcionário e o EPI");
        }

        // 2. O pedido chega só com os ids: busca os dados completos no banco
        Funcionario funcionario = funcionarioRepository
                .findById(pedido.getFuncionario().getId())
                .orElseThrow(() ->
                        new IllegalArgumentException("Funcionário não encontrado"));

        Epi epi = epiRepository
                .findById(pedido.getEpi().getId())
                .orElseThrow(() ->
                        new IllegalArgumentException("EPI não encontrado"));

        // 3. Regras de negócio (NR-06)
        if (epi.isCaVencido()) {
            throw new IllegalArgumentException(
                    "EPI com CA vencido não pode ser emprestado");
        }

        if (epi.getQuantidadeDisponivel() <= 0) {
            throw new IllegalArgumentException("EPI sem estoque disponível");
        }

        boolean jaSolicitou = emprestimoRepository
                .existsByFuncionarioIdAndEpiIdAndStatus(
                        funcionario.getId(), epi.getId(), "SOLICITADO");

        if (jaSolicitou) {
            throw new IllegalArgumentException(
                    "Você já tem uma solicitação aberta para este EPI");
        }

        // 4. A API decide estes campos (não confiamos no que veio do app)
        pedido.setId(null);                 // sempre um registro novo
        pedido.setFuncionario(funcionario); // troca o "só id" pelo objeto completo
        pedido.setEpi(epi);
        pedido.setDataEmprestimo(LocalDate.now());
        pedido.setDataDevolucao(null);
        pedido.setStatus("SOLICITADO");

        // 5. Grava na tabela emprestimo
        return emprestimoRepository.save(pedido);
    }

    public List<Emprestimo> listarPorFuncionario(Long funcionarioId) {
        return emprestimoRepository.findByFuncionarioIdOrderByIdDesc(funcionarioId);
    }
}
```

Explicação - O Service guarda as regras de negócio. O Controller só recebe e responde.

Antes de gravar, o método solicitar() confere:

Situação	Mensagem devolvida
Faltou o id do funcionário ou do EPI	Informe o funcionário e o EPI
Id que não existe no banco	Funcionário não encontrado / EPI não encontrado
CA vencido (ex.: óculos, id 6)	EPI com CA vencido não pode ser emprestado
Quantidade igual a zero (ex.: concha, id 3)	EPI sem estoque disponível
Mesmo funcionário já pediu o mesmo EPI	Você já tem uma solicitação aberta para este EPI

Repare que a consulta `existsByFuncionarioIdAndEpiIdAndStatus` foi criada no EmprestimoRepository lá na Aula 1. Hoje ela finalmente é usada.

Passo 4 do método (linhas 69 a 75): a API, e não o app, decide o id, a data, o status e troca o funcionário e o EPI "só com id" pelos objetos completos vindos do banco.

O estoque **não** diminui aqui: o pedido ainda não foi aprovado. Isso fica para a aula de aprovação.

Etapa 4 – Criar o EmprestimoController

Pacote: `com.senac.api_epi.controllers`

Criar a classe: EmprestimoController

EmprestimoController.java

```java
package com.senac.api_epi.controllers;

import com.senac.api_epi.entities.Emprestimo;
import com.senac.api_epi.service.EmprestimoService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/emprestimos")
public class EmprestimoController {

    private final EmprestimoService service;

    public EmprestimoController(EmprestimoService service) {
        this.service = service;
    }

    // POST /emprestimos -> funcionário solicita um EPI
    @PostMapping
    public ResponseEntity<?> solicitar(@RequestBody Emprestimo pedido) {
        try {
            Emprestimo emprestimo = service.solicitar(pedido);
            return ResponseEntity
                    .status(HttpStatus.CREATED) // 201
                    .body(emprestimo);
        } catch (IllegalArgumentException e) {
            return ResponseEntity
                    .badRequest()               // 400
                    .body(e.getMessage());
        }
    }

    // GET /emprestimos/funcionario/2 -> empréstimos de um funcionário
    @GetMapping("/funcionario/{id}")
    public List<Emprestimo> listarPorFuncionario(@PathVariable Long id) {
        return service.listarPorFuncionario(id);
    }
}
```

Explicação

- `POST /emprestimos` recebe o JSON e o transforma em um objeto Emprestimo (`@RequestBody`), como no login da Aula 3.
- Deu certo → responde **201 Created** com o empréstimo gravado.
- Regra quebrada → responde **400 Bad Request** com a mensagem do Service.
- `GET /emprestimos/funcionario/{id}` lista os pedidos de um funcionário. Vamos usar para conferir e no exercício.

Fluxo da solicitação na API

Flutter
↓
POST /emprestimos
↓
EmprestimoController
↓
EmprestimoService (regras)
↓
EmprestimoRepository
↓
Tabela emprestimo → status SOLICITADO

Etapa 5 – Testando no Postman

Reinicie a API antes de testar.

Teste 1 – Solicitação válida

POST http://localhost:8080/emprestimos

Body → raw → JSON:

```
{
  "funcionario": { "id": 2 },
  "epi": { "id": 1 },
  "observacao": "Capacete atual está trincado"
}
```

Resposta esperada: status **201 Created**

```
{
  "id": 1,
  "funcionario": {
    "id": 2,
    "matricula": "1002",
    "nome": "Bruno Lima",
    "setor": "Almoxarifado",
    "perfil": "FUNCIONARIO"
  },
  "epi": {
    "id": 1,
    "nome": "Capacete de segurança classe B",
    "categoria": "Proteção da Cabeça",
    "ca": "40101",
    "validadeCa": "2028-03-15",
    "imagemUrl": "/imagens/capacete.jpg",
    "quantidadeDisponivel": 10,
    "caVencido": false
  },
  "dataEmprestimo": "(data de hoje)",
  "dataDevolucao": null,
  "status": "SOLICITADO",
  "observacao": "Capacete atual está trincado"
}
```

Observe: mandamos o funcionário e o EPI só com o id, mas a resposta traz os dois completos. É o relacionamento @ManyToOne funcionando. E a senha continua sem aparecer.

Teste 2 – CA vencido: troque para `"epi": { "id": 6 }` (óculos)

Resposta: 400 → EPI com CA vencido não pode ser emprestado

Teste 3 – Sem estoque: troque para `"epi": { "id": 3 }` (protetor tipo concha)

Resposta: 400 → EPI sem estoque disponível

Teste 4 – Pedido repetido: envie de novo o Teste 1

Resposta: 400 → Você já tem uma solicitação aberta para este EPI

Teste 5 – Listar os pedidos do Bruno

GET http://localhost:8080/emprestimos/funcionario/2

Conferindo no MySQL Workbench

Use JOIN para juntar as três tabelas pelo relacionamento:

```
SELECT e.id, f.nome AS funcionario, p.nome AS epi,
       e.status, e.data_emprestimo, e.observacao
FROM emprestimo e
JOIN funcionario f ON f.id = e.funcionario_id
JOIN epi p ON p.id = e.epi_id;
```

Resultado esperado:

```
1 | Bruno Lima | Capacete de segurança classe B | SOLICITADO | (data) | Capacete atual...
```

Etapa 6 – Conferindo o model Epi no Flutter

Arquivo: models/epi.dart

A tela de detalhes precisa da quantidade, da validade e do campo caVencido. Se você fez o exercício da Aula 2, seu model já está assim. Confira:

epi.dart

```dart
class Epi {
  final int id;
  final String nome;
  final String categoria;
  final String ca;
  final String imagemUrl;
  final int quantidadeDisponivel; // Exercício 3a
  final DateTime validadeCa;      // Exercício 3b
  final bool caVencido;           // Exercício 3b

  Epi({
    required this.id,
    required this.nome,
    required this.categoria,
    required this.ca,
    required this.imagemUrl,
    required this.quantidadeDisponivel,
    required this.validadeCa,
    required this.caVencido,
  });

  // Construtor de fábrica que converte um JSON recebido da API
  // em um objeto Epi do Flutter.
  // Cada campo do JSON é associado ao atributo correspondente da classe.
  factory Epi.fromJson(Map<String, dynamic> json) {
    // A API envia a data como texto: "2028-03-15"
    final validade = DateTime.parse(json['validadeCa']);

    return Epi(
      id: json['id'],
      nome: json['nome'],
      categoria: json['categoria'],
      ca: json['ca'],
      imagemUrl: json['imagemUrl'],
      quantidadeDisponivel: json['quantidadeDisponivel'],
      validadeCa: validade,
      // O método isCaVencido() do Java vira o campo "caVencido" no JSON.
      // Se por algum motivo ele não vier, o app calcula pela data.
      caVencido: json['caVencido'] ?? validade.isBefore(DateTime.now()),
    );
  }

  // Data no formato brasileiro: 30/06/2025
  String get validadeFormatada {
    final dia = validadeCa.day.toString().padLeft(2, '0');
    final mes = validadeCa.month.toString().padLeft(2, '0');
    return '$dia/$mes/${validadeCa.year}';
  }
}
```

Etapa 7 – Criando o EmprestimoService

Arquivo: services/emprestimo_service.dart

emprestimo_service.dart

```dart
import 'dart:convert';
import 'package:http/http.dart' as http;
import 'api_config.dart';

class EmprestimoService {

  // Envia a solicitação de empréstimo para a API
  Future<void> solicitar({
    required int funcionarioId,
    required int epiId,
    required String observacao,
  }) async {
    final response = await http.post(
      Uri.parse('${ApiConfig.baseUrl}/emprestimos'),
      headers: {
        'Content-Type': 'application/json',
      },
      // Mesmo formato da entidade Emprestimo do Java:
      // funcionário e EPI vão "dentro" do pedido, só com o id
      body: jsonEncode({
        'funcionario': {'id': funcionarioId},
        'epi': {'id': epiId},
        'observacao': observacao,
      }),
    );

    // 201 = Created: o empréstimo foi gravado
    if (response.statusCode == 201) {
      return;
    }

    // 400: a API explica o motivo (CA vencido, sem estoque...)
    throw Exception(utf8.decode(response.bodyBytes));
  }
}
```

O que esta classe faz?

É parecida com o AuthService da Aula 3:

- monta o JSON com `jsonEncode`, no formato aninhado da Etapa 2;
- envia com `http.post` e o header `Content-Type: application/json`;
- se a API responder **201**, terminou;
- se não, lança uma Exception com a mensagem que a API mandou (CA vencido, sem estoque...).

O `utf8.decode(response.bodyBytes)` garante que os acentos da mensagem cheguem certos.

Etapa 8 – Criando a tela EpiDetalheScreen

Arquivo: screens/epi_detalhe_screen.dart

A tela é grande, por isso está dividida em partes. A numeração das linhas continua de uma imagem para a outra.

epi_detalhe_screen.dart

```dart
import 'package:flutter/material.dart';
import '../models/epi.dart';
import '../models/usuario.dart';
import '../services/api_config.dart';
import '../services/emprestimo_service.dart';

class EpiDetalheScreen extends StatefulWidget {
  final Epi epi;          // EPI escolhido na lista
  final Usuario usuario;  // funcionário logado

  const EpiDetalheScreen({
    super.key,
    required this.epi,
    required this.usuario,
  });

  @override
  State<EpiDetalheScreen> createState() =>
      _EpiDetalheScreenState();
}

class _EpiDetalheScreenState
    extends State<EpiDetalheScreen> {
  final formKey = GlobalKey<FormState>();
  final observacaoController = TextEditingController();
  bool carregando = false;

  @override
  void dispose() {
    observacaoController.dispose();
    super.dispose();
  }
```

epi_detalhe_screen.dart

```dart
  Future<void> solicitarEmprestimo() async {
    // 1. Valida o formulário antes de chamar a API
    if (!formKey.currentState!.validate()) return;

    setState(() {
      carregando = true;
    });

    try {
      // 2. Envia o POST
      await EmprestimoService().solicitar(
        funcionarioId: widget.usuario.id,
        epiId: widget.epi.id,
        observacao: observacaoController.text.trim(),
      );

      if (!mounted) return;

      // 3. Sucesso: avisa e volta para a lista
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(
          content: Text('Solicitação enviada! Aguarde a aprovação.'),
          backgroundColor: Colors.green,
        ),
      );
      Navigator.pop(context);
    } catch (e) {
      if (!mounted) return;

      // 4. Erro: mostra a mensagem que veio da API
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: Text(
            e.toString().replaceFirst('Exception: ', ''),
          ),
          backgroundColor: Colors.red,
        ),
      );
    } finally {
      if (mounted) {
        setState(() {
          carregando = false;
        });
      }
    }
  }
```

epi_detalhe_screen.dart

```dart
  @override
  Widget build(BuildContext context) {
    final epi = widget.epi;

    // O app só libera o botão se o EPI puder ser emprestado
    final podeSolicitar =
        !epi.caVencido && epi.quantidadeDisponivel > 0;

    return Scaffold(
      appBar: AppBar(
        title: const Text('Detalhes do EPI'),
        backgroundColor: Colors.blue,
        foregroundColor: Colors.white,
      ),
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(20),
        child: Form(
          key: formKey,
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              // Foto do EPI
              ClipRRect(
                borderRadius: BorderRadius.circular(12),
                child: Image.network(
                  '${ApiConfig.baseUrl}${epi.imagemUrl}',
                  height: 200,
                  fit: BoxFit.contain,
                  errorBuilder: (context, error, stackTrace) {
                    return Container(
                      height: 200,
                      color: Colors.blue.shade50,
                      child: const Icon(
                        Icons.health_and_safety,
                        size: 80,
                        color: Colors.blue,
                      ),
                    );
                  },
                ),
              ),
              const SizedBox(height: 20),
```

epi_detalhe_screen.dart

```dart
              // Dados do EPI
              Text(
                epi.nome,
                style: const TextStyle(
                  fontSize: 22,
                  fontWeight: FontWeight.bold,
                ),
              ),
              const SizedBox(height: 6),
              Text(
                epi.categoria,
                style: const TextStyle(
                  fontSize: 16,
                  color: Colors.grey,
                ),
              ),
              const Divider(height: 30),
              linhaInfo(Icons.verified, 'CA', epi.ca),
              linhaInfo(
                Icons.event,
                'Validade do CA',
                epi.validadeFormatada,
                cor: epi.caVencido ? Colors.red : null,
              ),
              linhaInfo(
                Icons.inventory_2,
                'Disponível',
                '${epi.quantidadeDisponivel} un.',
              ),
              const Divider(height: 30),

              // Formulário de solicitação
              Text(
                'Solicitante: ${widget.usuario.nome} '
                '(matrícula ${widget.usuario.matricula})',
              ),
```

epi_detalhe_screen.dart

```dart
              const SizedBox(height: 16),
              TextFormField(
                controller: observacaoController,
                maxLines: 3,
                maxLength: 200,
                decoration: const InputDecoration(
                  labelText: 'Motivo da solicitação',
                  hintText: 'Ex.: capacete atual está trincado',
                  border: OutlineInputBorder(),
                ),
                validator: (valor) {
                  if (valor == null || valor.trim().isEmpty) {
                    return 'Informe o motivo da solicitação';
                  }
                  if (valor.trim().length < 5) {
                    return 'Escreva pelo menos 5 caracteres';
                  }
                  return null; // null = campo válido
                },
              ),
              const SizedBox(height: 8),

              // Aviso quando o EPI não pode ser emprestado
              if (!podeSolicitar)
                Padding(
                  padding: const EdgeInsets.only(bottom: 8),
                  child: Text(
                    epi.caVencido
                        ? 'CA vencido: este EPI não pode ser emprestado.'
                        : 'EPI sem estoque no momento.',
                    textAlign: TextAlign.center,
                    style: const TextStyle(
                      color: Colors.red,
                      fontWeight: FontWeight.bold,
                    ),
                  ),
                ),
```

epi_detalhe_screen.dart

```dart
              // Botão Solicitar Empréstimo
              SizedBox(
                height: 50,
                child: ElevatedButton.icon(
                  onPressed: (carregando || !podeSolicitar)
                      ? null
                      : solicitarEmprestimo,
                  icon: carregando
                      ? const SizedBox(
                          width: 20,
                          height: 20,
                          child: CircularProgressIndicator(
                            strokeWidth: 2,
                            color: Colors.white,
                          ),
                        )
                      : const Icon(Icons.send),
                  label: const Text('Solicitar Empréstimo'),
                  style: ElevatedButton.styleFrom(
                    backgroundColor: Colors.blue,
                    foregroundColor: Colors.white,
                  ),
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }

  // Monta uma linha "ícone + rótulo + valor"
  Widget linhaInfo(IconData icone, String rotulo, String valor,
      {Color? cor}) {
    return Padding(
      padding: const EdgeInsets.symmetric(vertical: 6),
      child: Row(
        children: [
          Icon(icone, color: cor ?? Colors.blue),
          const SizedBox(width: 10),
          Text(
            '$rotulo: ',
            style: const TextStyle(fontWeight: FontWeight.bold),
          ),
          Expanded(
            child: Text(valor, style: TextStyle(color: cor)),
          ),
        ],
      ),
    );
  }
}
```

Explicando o código

Parâmetros da tela (linhas 7 a 15)

A tela recebe dois objetos no construtor: o `epi` escolhido e o `usuario` logado. Dentro do State eles são acessados com `widget.epi` e `widget.usuario`.

Formulário (linhas 24, 98 e 161 a 178)

- `GlobalKey<FormState>` é a "chave" que permite validar o formulário inteiro.
- `TextFormField` é um TextField que sabe se validar.
- O `validator` devolve um texto de erro ou `null` quando o campo está certo.
- `formKey.currentState!.validate()` roda todos os validators. Se algum falhar, a API nem é chamada.

Botão Solicitar Empréstimo (linhas 199 a 222)

- Fica desabilitado (`onPressed: null`) se o CA estiver vencido, sem estoque ou enquanto a API responde.
- Ao tocar, chama `solicitarEmprestimo()`, que valida, envia o POST e mostra o resultado.

SnackBar verde ou vermelha (linhas 52 a 70)

Sucesso: mostra a mensagem e volta para a lista com `Navigator.pop`. Erro: mostra a mensagem que veio da API.

Atenção: a regra está nos dois lados
O app desabilita o botão para ajudar o usuário, mas quem garante a regra é a API. Alguém poderia chamar a API pelo Postman sem passar pelo app, e a API recusaria do mesmo jeito.

Etapa 9 – Navegação com parâmetros: ajustando a EpiScreen

Arquivo: screens/epi_screen.dart

Para abrir os detalhes, a EpiScreen precisa conhecer o usuário logado. Veja as linhas alteradas (a numeração pode ser diferente no seu arquivo):

a) Imports e construtor (topo do arquivo)

epi_screen.dart

```dart
import 'package:flutter/material.dart';

import '../models/epi.dart';
import '../models/usuario.dart'; // Aula 4
import '../services/api_config.dart';
import '../services/epi_service.dart';
import 'epi_detalhe_screen.dart'; // Aula 4

class EpiScreen extends StatelessWidget {
  final Usuario usuario; // Aula 4: recebe o usuário logado

  const EpiScreen({
    super.key,
    required this.usuario, // Aula 4
  });
```

b) Dentro do ListTile, acrescente o onTap antes do leading

epi_screen.dart

```
71                  child: ListTile(
72                    // Aula 4: ao tocar no EPI, abre a tela de detalhes
73                    onTap: () {
74                      Navigator.push(
75                        context,
76                        MaterialPageRoute(
77                          builder: (_) => EpiDetalheScreen(
78                            epi: epi,
79                            usuario: usuario,
80                          ),
81                        ),
82                      );
83                    },
84
85                    leading: ClipRRect(
```

O `onTap` faz o card responder ao toque. `EpiDetalheScreen(epi: epi, usuario: usuario)` é a navegação com parâmetros: estamos entregando os dados para a próxima tela.

Etapa 10 – Ajustando a MainScreen

Arquivo: screens/main_screen.dart

Depois da Etapa 9 o VS Code vai marcar erro no menu: a EpiScreen agora exige o usuário. No ListTile de EPIs, troque:

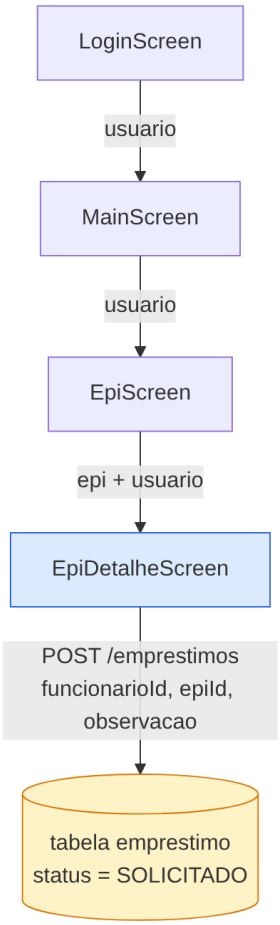
```
builder: (_) => const EpiScreen(),
```

por:

```
builder: (_) => EpiScreen(usuario: usuario), // Aula 4
```

Por que tirar o const? O `usuario` só é conhecido quando o app está rodando (depois do login). Um widget const precisa ter tudo definido antes de rodar.

O caminho do usuário pelo app



Etapa 11 – Rodando e testando

Deixe a API e o MySQL rodando e execute o app. Faça este roteiro:

1. Entre com a matrícula 1003 / senha 1234 (Carla). O Bruno já pediu o capacete no Postman.
2. Menu → EPIs → toque no Capacete.
3. Toque em Solicitar Empréstimo sem escrever nada: o formulário mostra o erro em vermelho.
4. Escreva o motivo e toque de novo: SnackBar verde e volta para a lista.
5. Abra o Capacete e solicite outra vez: SnackBar vermelha com a mensagem da API.
6. Abra os Óculos (CA vencido) e o Protetor tipo concha (sem estoque): o botão fica desabilitado.
7. Rode o SELECT com JOIN no Workbench e veja o pedido da Carla.

Possíveis erros e como resolver

Erro	Causa provável / solução
The named parameter 'usuario' is required	A MainScreen ainda chama EpiScreen() sem o usuário. Veja a Etapa 10.
Erro no const da MainScreen	Tire o const de EpiScreen(usuario: usuario).
400 – Informe o funcionário e o EPI	O JSON não está no formato aninhado: use "funcionario": {"id": ...} e "epi": {"id": ...}.
415 Unsupported Media Type	Faltou o header 'Content-Type': 'application/json'.
Unknown column 'observacao'	Reinicie a API ou rode o ALTER TABLE da Parte 1.
ClientException / Failed to fetch	A API não está rodando ou o endereço do ApiConfig está errado.
Acentos estranhos na mensagem	Use utf8.decode(response.bodyBytes) no service.

O fluxo da aplicação está nesta ordem:

EpiScreen
↓
EpiDetalheScreen (epi + usuario)
↓
Formulário validado
↓
EmprestimoService (Flutter)
↓
POST /emprestimos
↓
EmprestimoController → EmprestimoService (Java)
↓
Tabela emprestimo (SOLICITADO)

Exercício

1) Crie a tela "Meus Empréstimos" e coloque no menu lateral.

Requisitos

- a) Criar o model Emprestimo no Flutter com os campos id, dataEmprestimo, status, observacao e um `Epi epi`.
- b) No fromJson, converter o EPI que vem dentro do JSON: `epi: Epi.fromJson(json['epi'])`.
- c) Criar no EmprestimoService o método `buscarPorFuncionario(int id)` usando GET /emprestimos/funcionario/{id}.
- d) Exibir: nome do EPI, data da solicitação, motivo e status.
- e) Mostrar só os empréstimos do usuário logado.

2) Antes de enviar, mostre um AlertDialog de confirmação:

```
Confirma a solicitação de "Capacete de segurança classe B"?
                              [Cancelar]   [Confirmar]
```

3) Na tela "Meus Empréstimos", mostre o status com cor:

- SOLICITADO → laranja
- EMPRESTADO → verde
- DEVOLVIDO → cinza

Dica com IA: copie a resposta do Teste 5 do Postman e peça a um assistente de IA: "gere o model Dart com fromJson para este JSON". Depois compare com o seu model: os nomes batem? Ele tratou o EPI aninhado e a data?

Desafio extra: na API, impeça também um novo pedido quando o funcionário já estiver com o mesmo EPI em status EMPRESTADO (ainda não devolvido).